```
!pip install -qqqq -U -f https://pytorch.org/whl/torch_stable.html fastai fastai2
!apt install -qqq -y -q libsndfile1
!pip install -qqqq -U -q -f https://dl.fbaipublicfiles.com/fastai/fastai2/dev-builds/latest.zip fastai2[dev,nbdev]
!pip install -qqqq -U -q -f https://dl.fbaipublicfiles.com/fastai/fastai2/dev-builds/latest.zip fastai2[audio,dev,nbdev]
!pip install -qqqq -U -q -f https://dl.fbaipublicfiles.com/fastai/fastai2/dev-builds/latest.zip fastai2[text,dev,nbdev]
!pip install -qqqq -U -q -f https://dl.fbaipublicfiles.com/fastai/fastai2/dev-builds/latest.zip fastai2[vision,dev,nbdev]
!pip install -qqqq -U -q -f https://dl.fbaipublicfiles.com/fastai/fastai2/dev-builds/latest.zip fastai2[tabular,dev,nbdev]
!pip install -qqqq -U -q -f https://dl.fbaipublicfiles.com/fastai/fastai2/dev-builds/latest.zip fastai2[extra,dev,nbdev]
!pip install -qqqq -U -q -f https://dl.fbaipublicfiles.com/fastai/fastai2/dev-builds/latest.zip fastai2[all
```

```
    traditional_unique = len(np.unique(traditional_buckets))
    traditional_collision = 1 - traditional_unique / sample_size
    reduce_ratio = (traditional_collision - learned_collision) / traditional_collision) * 100 if traditional_col
    learned_bucket_coverage = learned_unique / hash_index.num_buckets * 100
    traditional_bucket_coverage = traditional_unique / hash_index.num_buckets * 100

    print(f"\n═══ Paper Evaluation Result (Unique Data Validation) ═══")
    print(f"Sample Size: {sample_size} | Number of Buckets: {hash_index.num_buckets}")
    print(f"Learned Hash: Collision Rate {learned_collision:.4f} | Bucket Coverage {learned_bucket_coverage:.1f}%
    print(f"MurmurHash: Collision Rate {traditional_collision:.4f} | Bucket Coverage {traditional_bucket_coverage
    print(f"Collision Rate Reduction: {reduce_ratio:.2f}% (Paper Target ≥25%)")

    return learned_buckets, traditional_buckets, reduce_ratio

def plot_result(learned_buckets, traditional_buckets, num_buckets):
    plt.rcParams['font.sans-serif'] = ['Arial']
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
    fig.suptitle('Learned Hash vs MurmurHash (Unique Data)', fontweight='bold')

    # 图1: 桶分布
    ax1.hist(learned_buckets, bins=30, alpha=0.7, label='Learned Hash', color='#1f77b4')
    ax1.hist(traditional_buckets, bins=30, alpha=0.7, label='MurmurHash', color='#ff7f0e')
    ax1.set_xlabel('Hash Bucket Index')
    ax1.set_ylabel('Number of Keys')
    ax1.legend()
    ax1.grid(alpha=0.3)

    # 图2: 冲突率对比
    learned_collision = 1 - len(np.unique(learned_buckets)) / len(learned_buckets)
    traditional_collision = 1 - len(np.unique(traditional_buckets)) / len(traditional_buckets)
    bars = ax2.bar(
        ['Learned Hash', 'MurmurHash'],
        [learned_collision, traditional_collision],
        color=['#1f77b4', '#ff7f0e'],
        width=0.6, alpha=0.8
    )
    for bar, val in zip(bars, [learned_collision, traditional_collision]):
        ax2.text(bar.get_x()+bar.get_width()/2, val+0.005, f'{val:.4f}', ha='center', fontweight='bold')
    ax2.set_ylabel('Collision Rate (Lower = Better!)')
    ax2.set_ylim(0, max(learned_collision, traditional_collision) * 1.2)
    ax2.grid(alpha=0.3, axis='y')

    plt.tight_layout()


if __name__ == "__main__":
    try:
        # 生成数据集
        print("=== Generating Log-Normal Dataset ===")
        data = generate_paper_data(CONFIG["target_data_size"])

        # 训练学习型哈希模型
        print("\n=== Training RMI Learned Hash ===")
        learned_hash = PaperLearnedHash()
        learned_hash.train(data)

        # 评估性能
        learned_buckets, traditional_buckets, reduce_ratio = evaluate_result(learned_hash, data)

        # 可视化
        plot_result(learned_buckets, traditional_buckets, learned_hash.num_buckets)

    except ValueError as e:
        print(f"\n数据集生成失败: {e}")
```

```
=== Generating Log-Normal Dataset ===
Generated Dataset:10,000 entries (Unique keys:10000?)
Data Range:[1000, 13568]

=== Training RMI Learned Hash ===
Number of Hash Buckets:8000 (Unique Data Size:10000)

=== Training Stage1 ===
Stage 1 completed (full 60 epochs).

=== Training Stage2 ===
Stage 2 Completed: 20/20 models trained effectively.

=== Paper Evaluation Result (Unique Data Validation) ===
Sample Size: 5000 | Number of Buckets: 8000
Learned Hash: Collision Rate 0.1692 | Bucket Coverage 51.9%
MurmurHash: Collision Rate 0.2574 | Bucket Coverage 46.4%
Collision Rate Reduction: 34.27% (Paper Target ≥25%)
```

In [3]:
```
# ==========================================
# Section 5: Learned Bloom Filter
# ==========================================
print("\n" + "=">50)
print("   SECTION 5: LEARNED BLOOM FILTER   ")
print("="*50)

import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
# 引入必要的归一化工具
from sklearn.preprocessing import MinMaxScaler

class LearnedBloomFilter:
    def __init__(self, threshold=0.6):
        """
        threshold: 阈值 tau, 模型打分 > tau 视为存在,
        """
        self.threshold = threshold
        self.model = self._build_classifier()
        # 备用过滤器，存储模型漏掉的 False Negatives
        self.backup_filter = set()
        self.estimated_model_size = 0 # unit: bits

        # 定义归一化器，将 Key 缩放到 0-1 之间
        # 神经网络在这里接收的是 0-1000万 这样的大整数
        self.scaler = MinMaxScaler()

    def _build_classifier(self):
        model = keras.Sequential([
            layers.Input(shape=(1,)),
            layers.Dense(32, activation='relu'),
            layers.Dense(16, activation='relu'),
            layers.Dense(1, activation='sigmoid') # 输出 0-1 的概率
        ])
        model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
        return model

    def train(self, positives, negatives):
        """
        positives: 实际上存在的 Key
        negatives: 实际上不存在的 Key
        """
        print("1. Preprocessing Data (Scaling)...")
        # 转换为 numpy 格式
        positives = np.array(positives, dtype=np.float32)
        negatives = np.array(negatives, dtype=np.float32)

        # 训练归一化器 (拟合数据范围)
        all_data = np.concatenate([positives, negatives]).reshape(-1, 1)
        self.scaler.fit(all_data)

        # 归一化数据 (变成 0-1 之间)
        X_pos = self.scaler.transform(positives.reshape(-1, 1))
        X_neg = self.scaler.transform(negatives[:len(positives)].reshape(-1, 1)) # 负采样

        # 构造训练集
        y_pos = np.ones(len(X_pos))
        y_neg = np.zeros(len(X_neg))

        X = np.concatenate([X_pos, X_neg])
        y = np.concatenate([y_pos, y_neg])

        # 打乱数据
        indices = np.arange(len(X))
        np.random.shuffle(indices)
        X = X[indices]
        y = y[indices]

        print("2. Training Classifier...")
        self.model.fit(X, y, epochs=10, batch_size=1024, verbose=0)

        # 3. 构建 Backup Filter
        print("3. Building Backup Filter...")
        # 预测所有 Positive Key
        preds = self.model.predict(X_pos, verbose=0).flatten()

        # 找出 False Negatives (被模型误判为不存在的)
        fn_indices = np.where(preds <= self.threshold)[0]

        # 注意：存入备用过滤器时要用原始 Key (整数)
        original_positives = positives.astype(np.int64)
        false_negatives = original_positives[fn_indices]

        for key in false_negatives:
            self.backup_filter.add(key)

        print(f"   Total Positives: {len(positives)}")
        print(f"   Model Missed (FN): {len(false_negatives)}")
        print(f"   Backup Filter Size: {len(self.backup_filter)} keys")

        # 估算模型权重大小
        total_weights = sum([np.prod(w.shape) for w in self.model.get_weights()])
        self.estimated_model_size = total_weights * 32

    def contains(self, key):
        # 查询前必须先归一化
        key_arr = np.array([[key]], dtype=np.float32)
        key_scaled = self.scaler.transform(key_arr)

        pred = self.model.predict(key_scaled, verbose=0)[0][0]
        if pred > self.threshold:
            return True

        # 查备用 (查原始 Key)
        if int(key) in self.backup_filter:
            return True
        return False

    def get_memory_usage_bits(self, target_fpr_backup=0.01):
        n = len(self.backup_filter)
        model_bits = self.estimated_model_size
        if n == 0: return model_bits
        # Bloom Filter 公式
        backup_bits = -(n * np.log(target_fpr_backup)) / (np.log(2)**2)
        return model_bits + backup_bits

def calc_optimal_bloom_filter_bits(n, p):
    return -(n * np.log(p)) / (np.log(2)**2)

# ==========================================
# Benchmark Execution
# ==========================================

# 1. 准备数据 (使用有规律的数据，模拟真实世界分布)
print("\nGenerating Structured Data (Learnable Patterns)...")
# 规则简单一个分布: Key 要么在 0-40% 要么 60-80% 的区间内
data_size = 1000000
all_data = np.arange(data_size)

mask_positive = ((all_data >= 0) & (all_data < 400000)) | \
                ((all_data >= 600000) & (all_data < 800000))

positive_keys = all_data[mask_positive]  # 存在的 Key
negative_keys = all_data[~mask_positive] # 不存在的 Key

# 打乱顺序
np.random.shuffle(positive_keys)
np.random.shuffle(negative_keys)

# 只取一部分进行快速测试 (例如 5万个)
N_TRAIN = 50000
train_positives = positive_keys[:N_TRAIN]
train_negatives = negative_keys[:N_TRAIN]

# 2. 训练
lbf = LearnedBloomFilter(threshold=0.6)
lbf.train(train_positives, train_negatives)

# 3. 评估结果
TARGET_FPR = 0.01
traditional_bits = calc_optimal_bloom_filter_bits(len(train_positives), TARGET_FPR)
learned_bits = lbf.get_memory_usage_bits(target_fpr_backup=TARGET_FPR)

# 4. 验证 FPR
print("\nMeasuring Actual False Positive Rate...")
# 用剩下的 Negative 数据测试
test_negatives = negative_keys[N_TRAIN:N_TRAIN+1000]
false_positives = 0
for k in test_negatives:
    if lbf.contains(k):
        false_positives += 1
actual_fpr = false_positives / len(test_negatives)
print(f"Actual FPR on Test Set: {actual_fpr:.4f}")

# 5. 结果对比
print("\n═══ Final Memory Comparison ═══")
print(f"Traditional Bloom Filter: {traditional_bits/8/1024:.2f} KB")
print(f"Learned Bloom Filter:     {learned_bits/8/1024:.2f} KB")
print(f"Space Saving:             {100 * (1 - learned_bits/traditional_bits):.2f}%")

plt.figure(figsize=(8, 5))
plt.bar(['Traditional BF', 'Learned BF'],
        [traditional_bits/8/1024, learned_bits/8/1024],
        color=['lightgray', '#2ca02c'])
plt.ylabel('Memory Usage (KB)')
plt.title('Bloom Filter Size Comparison\n(Actual FPR: {actual_fpr:.2%})')
for i, v in enumerate([traditional_bits/8/1024, learned_bits/8/1024]):
    plt.text(i, v, f"{v:.1f} KB", ha='center', va='bottom')
plt.show()
```

```
==================================================
   SECTION 5: LEARNED BLOOM FILTER
==================================================
Generating Structured Data (Learnable Patterns)...
1. Preprocessing Data (Scaling)...
2. Training Classifier...
3. Building Backup Filter...
   Total Positives: 50000
   Model Missed (FN): 16540
   Backup Filter Size: 16540 keys

Measuring Actual False Positive Rate...
Actual FPR on Test Set: 0.1400

═══ Final Memory Comparison ═══
Traditional Bloom Filter: 58.50 KB
Learned Bloom Filter:     21.68 KB
Space Saving:             62.93%
```

In [ ]: